

Camunda 8 Incident Resolution Guide

This document outlines standard operating procedures and troubleshooting steps for resolving the most common incidents in Camunda 8 (Zeebe, Operate, Tasklist, Connectors).

1. Job Worker & Service Task Failures (JOB_NO_RETRIES)

Symptoms

- An incident is raised in Operate on a Service Task or Business Rule Task.
- Incident message: **No retries left** or **Unhandled error raised during task execution**.
- Process instance state is set to **INCIDENT**.

Common Root Causes

1. **External Service Down / API Outage:** Target REST API or downstream microservice is unreachable.
2. **Data Parsing / Type Mismatch:** Unhandled exception or unexpected response payload structure.
3. **Worker Timeout:** Job exceeded its specified lock time (**timeout** setting) before completion.

Resolution Steps

1. **Investigate Logs:**
 - Open Operate, locate the incident, and examine the error message and stack trace.
 - Check worker logs (e.g., Spring Boot, Node.js, Python worker) for exact failure exceptions.

2. Fix Underlying Issue:

- If downstream service was offline, verify endpoint health and connectivity.
- If payload was invalid, update worker logic or fix incoming data via process instance modification.

3. Resolve Incident:

- In Operate, locate the job incident.
- Click **Increase Retries** (set to **1** or higher) or click **Retry Job**.
- Zeebe will re-dispatch the job to available workers.

2. FEEL Expression Evaluation Failures (CALCULATION_ERROR)

Symptoms

- Incident message: **Failed to evaluate expression '...' or Error during evaluation of FEEL expression.**
- Occurs at Gateways (Exclusive/Inclusive), Service Tasks (input/output mappings), or Multi-Instance loops.

Common Root Causes

1. **Missing Process Variable:** The expression references a variable that does not exist in the current scope.
2. **Null Pointer / Type Mismatch:** Accessing property on a null object (e.g., `request.user.id` when `user` is null) or operating on mismatched types (e.g., string vs number comparison).

Resolution Steps

1. **Identify Missing / Invalid Variable:**
 - Check the expression definition in the BPMN XML or Camunda Modeler.

- Inspect the process instance variables in Operate at the target element scope.
- 2. **Update Process Variables:**
 - In Operate, edit or add the missing variable in the **Variables** pane.
- 3. **Retry / Resolve:**
 - Click **Resolve Incident** in Operate to trigger re-evaluation of the FEEL expression.

3. Connector Executions & Authentication Failures

Symptoms

- Connector tasks (REST, HTTP, SendGrid, Kafka, Slack) raise incidents immediately upon execution.
- Incident message: **401 Unauthorized, 403 Forbidden, 404 Not Found**, or secret evaluation error.

Common Root Causes

1. **Expired / Invalid Secret:** API key, OAuth token, or basic auth credentials stored in Camunda Secrets / environment variables are expired or missing.
2. **Incorrect URL / Payload:** Connector payload mapping or target endpoint URL contains invalid syntax or invalid placeholders.

Resolution Steps

1. **Verify Secrets:**
 - Ensure the secret is defined correctly in the Connector environment / Secrets Store.
2. **Validate Request Setup:**
 - Test the target endpoint manually using cURL or Postman with the exact payload evaluated from Operate variables.
3. **Update Variables & Retry:**
 - Fix variable values in Operate if required.
 - Click **Retry** on the Connector incident.

4. Message Correlation Failures (Unmatched Message Events)

Symptoms

- Incoming message (from webhook, Kafka, or REST API) is published, but the process does not proceed.
- Intermediate Message Catch Event or Message Start Event fails to trigger, or message expires.

Common Root Causes

1. **Correlation Key Mismatch:** The correlation key defined in the BPMN message element does not match the payload parameter.
2. **Timing / Race Condition:** The message was published before the process instance reached the message catch boundary/state.
3. **TTL (Time-To-Live) Expired:** Message TTL expired before correlation occurred.

Resolution Steps

1. **Check Active Message Subscriptions:**
 - In Operate, view the process instance and check the active **Message Subscriptions** tab to verify expected message name and correlation key.
2. **Re-publish Message:**
 - Publish the message via Zeebe REST API ([POST /v2/messages/publication](#)) or gRPC API with the exact correlation key and payload.

5. Zeebe Resource & Backpressure Issues (RESOURCE_EXHAUSTED)

Symptoms

- Client applications or workers receive gRPC status `RESOURCE_EXHAUSTED` or HTTP status `429 Too Many Requests`.
- Cluster processing slows down or broker raft partitions lag behind.

Common Root Causes

1. **High Throughput / Spike in Load:** Processing volume exceeds cluster capacity/memory limits.
2. **Long-Running Transactions / Large Payloads:** Large variable payloads (>4MB default Zeebe limit) causing heavy I/O and serialization overhead.

Resolution Steps

1. **Apply Rate Limiting / Backoff:**
 - Ensure workers implement exponential backoff on retries.
2. **Scale Zeebe Cluster:**
 - Scale Zeebe broker replicas or increase CPU/Memory resources in Kubernetes / SaaS settings.
3. **Optimize Payload Sizes:**
 - Store large binaries/documents in external object storage (e.g., AWS S3, Google Cloud Storage) and pass references (URLs/IDs) inside process variables.

6. General Incident Resolution Checklist

1. **Locate in Operate:** Identify process instance ID and element ID.
2. **Identify Root Cause:** Distinguish between infrastructure outage, data error, or code defect.

3. **Fix Prerequisites:** Resolve external dependencies, code bugs, or payload variables first.
4. **Modify Variables (if required):** Correct process variables directly from Operate.
5. **Resolve / Retry:** Click **Resolve Incident** or **Increase Retries** in Operate / REST API.
6. **Verify:** Confirm the process instance advances past the failed element.